

➤ IT 3005 - Data Mining - Complete Note

• What is Data?

Collection of raw facts, observations, measurements or information that can be used for analysis, decision - making or further processing. [**Data is raw information collected about something.**]

Ex: - Student Names, Student Marks, Temperature, Number of Customers

• Types of Data

No	Type	Meaning	Examples
01.	Qualitative Data	Describes qualities or characteristics	Gender, color, type
02.	Quantitative Data	Represents numerical values	Age, height, salary
03.	Structured Data	Organized in a fixed format, such as tables	Excel/database records
04.	Unstructured Data	Does not have a fixed structure	Images, videos, text
05.	Primary Data	Collected directly by the researcher	Survey responses
06.	Secondary Data	Obtained from an existing source	Government reports

• Data vs Information

No	Data	Information
01.	Data is raw facts and observations .	Information is processed and meaningful data .
02.	It may not have a clear meaning by itself.	It provides useful meaning and understanding.
03.	It is the input for processing.	It is the output after processing.
04.	Data can be numbers, text, images and etc.	Information is usually presented in a useful form for decision - making.
05.	Ex: - 60, 70, 80, 90	Ex: - The average mark is 75.

• Numerical Data vs Categorical Data

Data can be broadly divided into “**Numerical Data**” and “**Categorical Data**”.

01. Numerical Data (Quantitative Data)

Represents quantities or amounts and is expressed using numbers. It can be used for mathematical calculations.

Ex: - Age, Height, Salary, Number of Students, Temperature

• Types of Numerical Data

1. Discrete Data - Countable values, usually whole numbers.

Ex: - Number of Books, Number of Students

2. Continuous Data - Can take any value within a range.

Ex: - Height, Weight

02. Categorical Data (Qualitative Data)

Represents groups, labels or categories rather than quantities. Mainly used to classify or group observations.

Ex: - Gender, Colour, Blood Type, Department, Education Level

• Types of Categorical Data

1. Nominal Data - Categories have no natural order.

Ex: - Colour, Blood Group,

2. Ordinal Data - Categories have a meaningful order or ranking.

Ex: - Satisfaction Level, Education Level

• What is Data Mining?

Process of discovering useful patterns, relationships, trends and hidden information from large amounts of data to predict future data. [Finding useful knowledge or patterns from large datasets.]

Prepare Data in Data Warehouses => Data Mining [Cleaned & Selected]	[Applying Algorithms]	=> Discover Patterns [Trends and Groups]
--	-----------------------	---

• Main Tasks of Data Mining

1. **Classification** - Assign data to categories.
Ex: Classify an email as Spam or Not Spam.
2. **Clustering** - Group similar data together.
Ex: Group customers based on their purchasing behavior.
3. **Association** - Find relationships between items.
Ex: People who buy bread often buy milk.
4. **Prediction** - Predict future outcomes.
Ex: Predict next month's sales.
5. **Anomaly Detection** - Find unusual or abnormal data.
Ex: Detect unusual credit - card transactions.

• Data Mining Workflow

No	Stage	Steps
01.	Data Collection	Gather data from CSV files, databases, APIs, sensors and etc.
02.	Data Loading	Load the dataset into memory. [Ex: - Using Pandas]
03.	Data Exploration (EDA)	Understand the data using statistics and visualizations.
04.	Data Cleaning (Preprocessing)	Remove missing values, duplicates, outliers and noise.
05.	Feature Engineering	Create, modify or select useful features from the raw data.
06.	Data Transformation	Encode categorical variables, normalize, standardize, scale and etc.
07.	Feature Selection	Select the most informative features.
08.	Train - Test Split	Divide the dataset into training and testing sets.
09.	Model Training (Model Fitting)	Train a machine learning algorithm on the training data.
10.	Prediction (Inference)	Use the trained model to predict outputs for new or test data.
11.	Model Evaluation	Measure performance using metrics such as accuracy, precision and etc.
12.	Hyperparameter Tuning	Optimize model parameters using Grid Search, Random Search and etc.
13.	Cross - Validation	Validate the model on multiple splits to estimate generalization performance.
14.	Model Deployment	Deploy the trained model into a real application.
15.	Monitoring & Maintenance	Monitor model performance and retrain when necessary.

• Data Mining Process

Data → Cleaning → Processing → Data Mining → Patterns → Decision (Deployment and Implementation)
--

• Advantages of Data Mining

1. Finds hidden patterns - Discovers useful patterns and relationships in large datasets.
2. Better decision - making - Helps organizations make data - driven decisions.
3. Predicts future trends - Can predict customer behavior, sales, demand and etc.
4. Fraud detection - Helps identify unusual or fraudulent activities.
5. Customer understanding - Helps businesses understand customer preferences.
6. Saves time and cost - Automates the analysis of large amounts of data.

- **Disadvantages of Data Mining**

1. Privacy issues - Personal information may be misused.
2. High cost - Requires computing resources, software and skilled people.
3. Complexity - Processing and analyzing large datasets can be difficult.
4. Poor-quality data - Incorrect or incomplete data can produce incorrect results.
5. Security risks - The sensitive data can be exposed or attacked.
6. Misinterpretation - Discovered patterns may be misunderstood or used incorrectly.

- **Limitations of Data Mining**

1. Depends on data quality - The bad data produces unreliable results.
2. Cannot guarantee correct predictions - Patterns found in past data may not always apply to the future.
3. Data privacy limitations - Some data cannot legally or ethically be collected or used.
4. Large data requirements - Some techniques need large amounts of data to produce reliable results.
5. Computational requirements - Very large datasets can require significant processing power.
6. Hidden bias - Data may contain biases, which can lead to biased results.

- **What is a Data Warehouse? [Definitions]**

1. Centralized repository that stores large amounts of historical and integrated data from different sources mainly for analysis, reporting and decision - making.
2. Central place where an organization stores valuable data from different sources in an organized form for analysis and decision - making.
3. Single Source of Truth (SSOT) because data from different systems is collected, cleaned, integrated and stored consistently in one central system.

Ex: - Snowflake [Cloud Data Warehouse] / Amazon Redshift [AWS Cloud] / Google BigQuery [Google Cloud]

- **Main Features of Data Warehouse**

01. **Subject - Oriented:** - Data is organized around important business subjects rather than individual applications or processes.

Ex: - Customers, Sales, Products, Employees

02. **Integrated:** - Data comes from multiple sources and is combined into a consistent format.

Ex: - Sales Database + Website + Mobile App + CRM [Customer Relationship Management]

03. **Time - Variant:** - Stores historical data over time.

Ex: - Total Sales Per Year

04. **Non - Volatile:** - Generally not frequently changed or deleted like data in an operational database.

[Data is relatively stable; mainly read, not continuously updated.]

05. **Summarized (Subject to Summarization):** - Available at different levels of aggregation to make analysis easier. [Ex: - Detailed Sales are summarized as total monthly sales.]

- **Advantages of a Data Warehouse**

1. Better Decision - Making: - Provides reliable data for business decisions.
2. Single Source of Truth: - Combines data from different systems into a consistent central repository.
3. Historical Analysis: - Allows organizations to compare data across months, years and etc.
4. Faster Reporting: - Data is organized specifically for analytical queries and reporting.
5. Data Integration: - Combines data from multiple sources.
6. Improved Data Quality: - Data can be cleaned and standardized before being stored.
7. Supports Business Intelligence: - Provides data for Dashboards, Reports, Data analysis, Data mining

• Disadvantages of a Data Warehouse

1. High Cost: - Building and maintaining a warehouse can be expensive.
2. Complex to Build: - Requires careful design, integration, ETL/ELT processes and skilled professionals. [ETL stands for Extract → Transform → Load / ELT stands for Extract → Load → Transform]
3. Time – Consuming: - Initial development and data migration can take considerable time.
4. Maintenance Required: - The warehouse must be maintained as business requirements and source systems change.
5. Data Integration Problems: - Different source systems may have inconsistent formats or definitions.
6. Large Storage Requirements: - Historical data can require significant storage.

• Limitations of a Data Warehouse

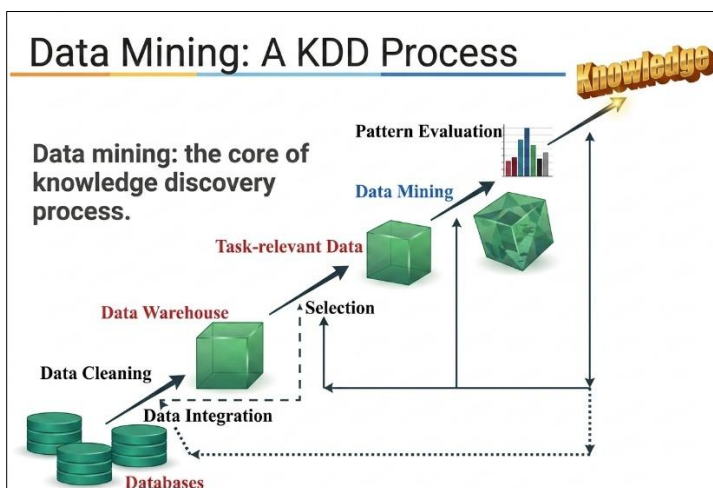
1. Not Usually Real - Time: - Traditional data warehouses may not contain the most recent data immediately.
2. Depends on Source Data: - If the original data is incorrect, the warehouse may also contain incorrect data.
3. Data Latency: - Delay between data being generated in an operational system and appearing in the warehouse.
4. Difficult to Change: - Changing the warehouse architecture after implementation can be complicated.
5. Security and Privacy Risks: - A centralized system containing valuable organizational data needs strong security.

• Applications of Data Warehouse

1. Business Intelligence (BI): - Helps businesses analyze data and make better decisions.
2. Sales Analysis: - Analyze sales by product, region, customer and time.
3. Marketing Analysis: - Understand customer behavior and measure marketing campaigns.
4. Customer Analysis: - Identify customer preferences, buying patterns, and valuable customers.
5. Financial Analysis: - Analyze revenue, expenses, profit and financial trends.
6. Inventory Management: - Monitor stock levels, product demand and inventory trends.

• KDD (Knowledge Discovery Process)

The process of finding useful, meaningful and previously unknown patterns or knowledge from large amounts of data.



1. **Database:** - Have unrecognized data coming from different sources.

2. **Cleaning & Integration:** - Errors are removed, Missing Values are handled, Data from multiple sources is combined.

3. **Data Warehouse:** - The storage of cleaned data.

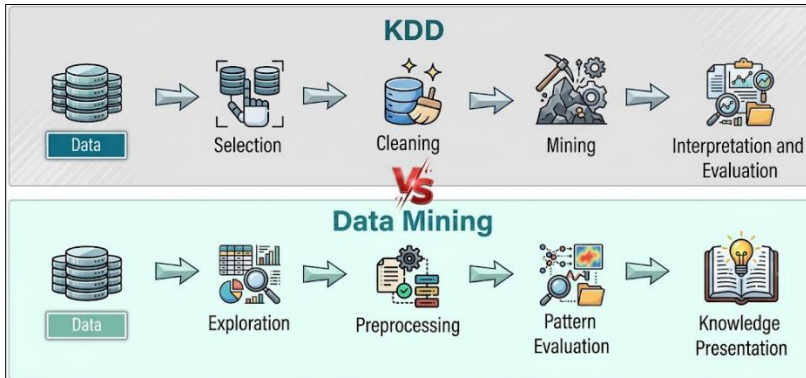
4. **Selection & Transformation:** - Only the useful part is prepared for Data Mining

5. **Data Mining (Core Step):** - Heart of the process.

• KDD Process

Step	Name	Simple Meaning
01.	Data Selection	Select the relevant data.
02.	Data Preprocessing	Clean the data and handle missing / noisy data.
03.	Data Transformation	Convert data into a suitable format.
04.	Data Mining	Find patterns and relationships.
05.	Pattern Evaluation	Identify useful and meaningful patterns.
06.	Knowledge Presentation	Present the discovered knowledge in an understandable way.

- **KDD vs Data Mining**



1.KDD => The complete knowledge discovery process.

2.Data Mining => One important step within KDD.

- **Data Visualization Techniques and Python**

“Data Visualization” is the process of representing data graphically using charts, graphs and other visual methods so that data is easier to understand and analyze.

In statistics, “Relationship” means how two or more variables are connected to each other. Understanding these relationships helps us analyze data and draw conclusions about a situation or population.

- **Why is Data Visualization Important?**

1. Easy to Understand: - Large amounts of data can be understood quickly.
2. Finds Patterns: - Helps identify patterns and trends that may be difficult to see in raw data.
3. Finds Outliers: - Unusual or abnormal values can be easily identified.
4. Helps Decision - Making: - Managers and analysts can make better decisions based on visual information.
5. Makes Comparisons Easy: - It is easier to compare different categories using charts.
6. Communicates Results: - Makes complex analysis easier to explain to others.

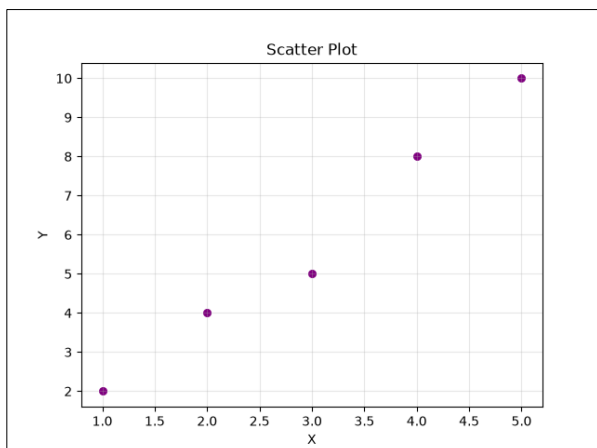
- **Common Data Visualization Techniques**

01. Scatter Plot

- Shows the relationship between two numerical variables.
- Helps to identify trends, patterns or correlation.

Ex: - Height vs Weight

- **Example For Chart**

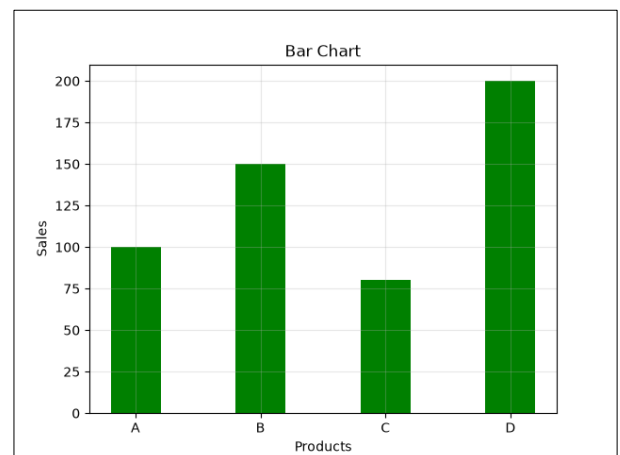


05. Bar Plot

- Uses to compare categorical variables.
- Can be vertical or horizontal.

Ex: - Number of Students in Each Department

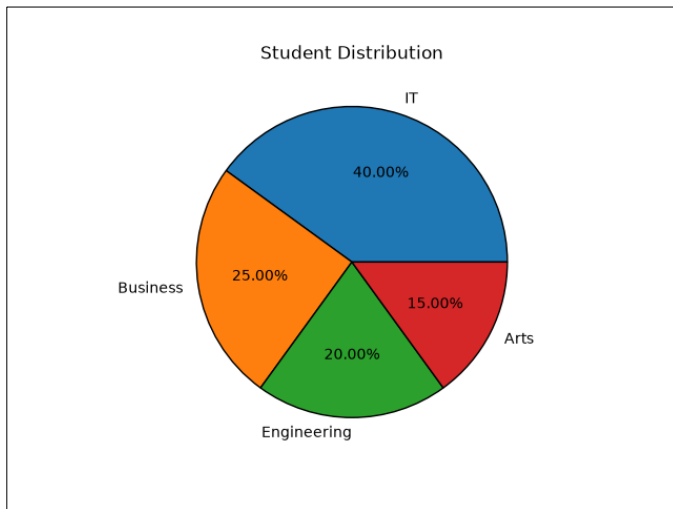
- **Example For Chart**



02. Pie Chart

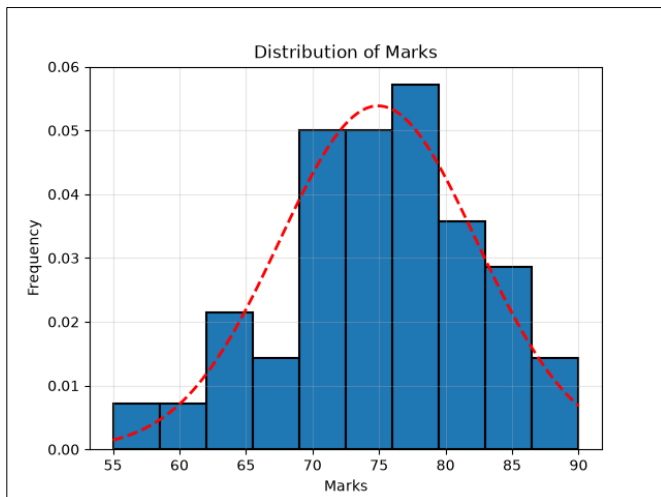
- Shows proportion of the percentages.
Ex: - Percentage of students by department

- **Example For Chart**



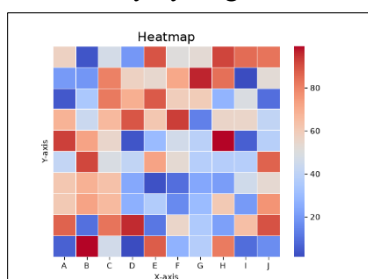
03. Histogram

- Shows distribution of numerical data.
- Groups data into intervals. (bins)
- Can identify Center, Spread, Skewness and Outliers
Ex: - Distribution of Exam Marks



04. Heatmap

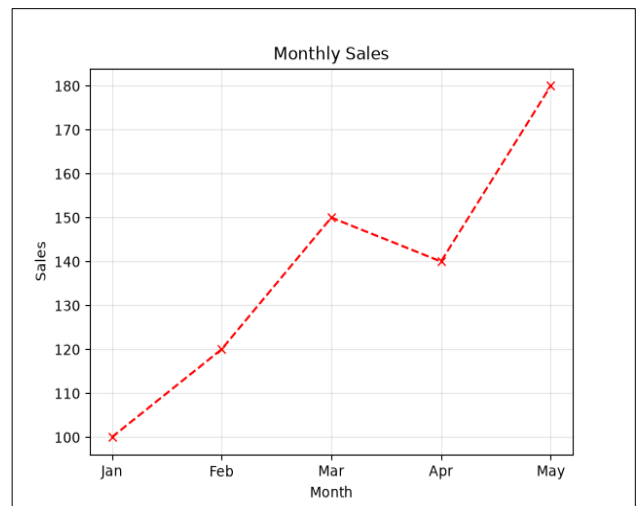
- Represents values using different shades / intensities.
- Common Use: Showing a correlation matrix.
Ex: - Population Density by Region



06. Line Chart

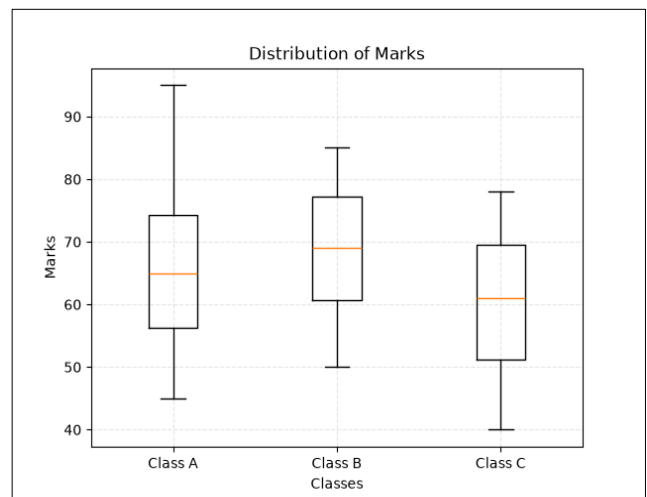
- Shows trends or changes over time.
Ex: - Daily Temperature

- **Example For Chart**



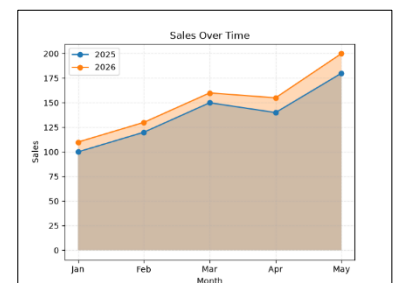
07. Box Plot

- Shows the distribution, spread, median, quartiles and outliers of numerical data.
Ex: - Salary Distribution



08. Area Chart

- Similar to a line chart but the area below the line is filled.
- Showing trends and the magnitude(volume) of values over time.
Ex: - Population Growth of Different Countries



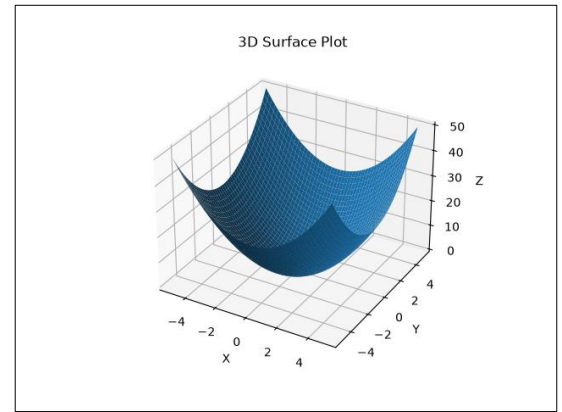
• What is a 3D Plot?

- A visualization that represents data using three axes: X, Y and Z.
- To visualize the relationship between three variables at the same time.

Example: -

Suppose we have student data;

- X-axis → Study Hours
- Y-axis → Attendance
- Z-axis → Exam Marks



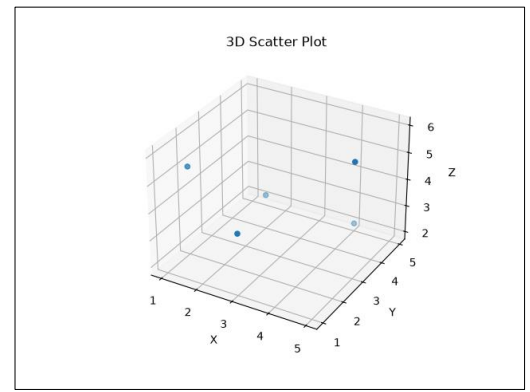
A 3D plot can help us visually understand how study hours and attendance are related to exam marks.

• Types of 3D Plots

1. **3D Scatter Plot** - Shows relationships between three numerical variables.
2. **3D Line Plot** - Shows a path or change in 3 dimensions.
3. **3D Surface Plot** - Shows a surface based on X, Y and Z values.

• Uses of 3D Plots

1. Understand Relationships
 - Helps visualize relationships between features and target variables.
2. Feature Analysis
 - Can help understand how multiple features interact.
3. Classification
 - With two features and one target dimension, 3D visualization can help understand different classes and their separation.
4. Model Understanding
 - Can help visualize a model's predictions or decision boundaries/surfaces in simple low-dimensional cases.



• Artificial Intelligence (AI)

Branch of Computer Science that focuses on building smart and intelligent machines capable of performing tasks that normally require human intelligence. [**Ability to work as people by independent thinking. (Mimic real human behaviors)**]

- **Tasks:** Learning, Reasoning, Problem - Solving, Understanding Language, Recognizing Images & Making Decisions

Ex: -

1. Google Assistant
2. Autonomous Cars

• Machine Learning (ML)

Subset of Artificial Intelligence (AI) that enables machines to learn from data without being explicitly programmed. It focuses on using data to make predictions and necessary decisions automatically.

Instead of writing rules manually: **Data → ML Algorithm → Learn Pattern → Prediction / Decision**

Ex: -

1. Image Classification [Features need to be extracted from an image before feeding it into a traditional ML algorithm. CNN (Convolutional Neural Network) can take images directly as input, automatically extract relevant features and make predictions using deep learning.]

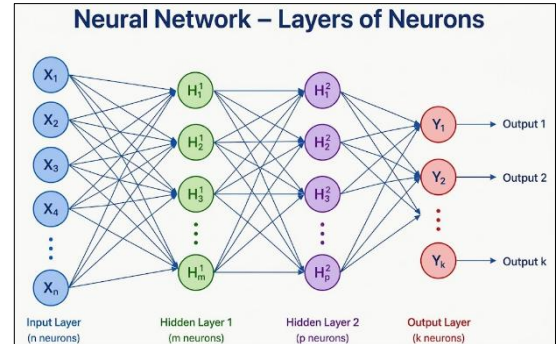
• Deep Learning (DL)

Subset of Machine Learning that uses artificial neural networks with multiple layers to automatically learn complex patterns from large amounts of data. [Follows neurons in our brain] The main components are:

1. **Input Layer** - Usually 1
2. **Hidden Layers** - 2 or more
3. **Output Layer** - Usually 1

Ex: -

1. Face recognition → Recognizes people from images.
2. Speech recognition → Converts speech into text.



• Artificial Neural Network (ANN)

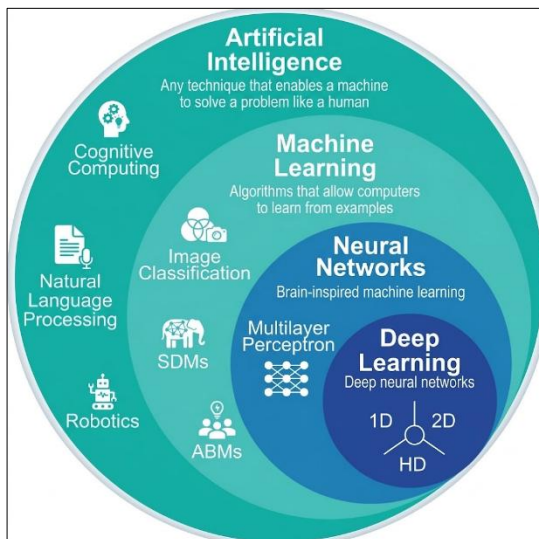
Computational model inspired by the way biological neurons work. It consists of connected neurons / nodes arranged in layers that process input data and learn patterns.

- Basic Structure: **Input Layer** → **Hidden Layer(s)** → **Output Layer**

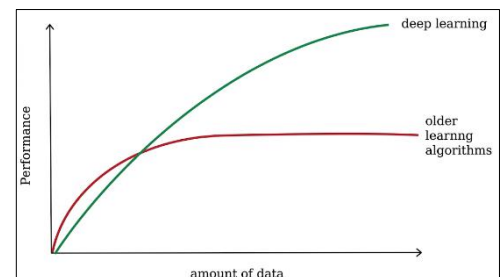
ANNs learn by passing data through interconnected neurons using weights and biases to transform the data and adjusting those weights during training so that the network can make better predictions.

“**Neuron**” is the basic processing unit of an Artificial Neural Network (ANN). It receives inputs, applies weights and a bias and produces an output. The number of neurons can be different in each layer and one layer can contain many neurons.

- Process: **Take Inputs / Process the Data / Carries the Process Data / Output**
- **Comparison of AI, ML, ANN, and Deep Learning (DL)**



1. Machine Learning (ML) is a subset of Artificial Intelligence (AI).
2. Deep Learning (DL) is a subset of Machine Learning (ML).
3. Deep Learning generally performs well with large amounts of data while traditional ML methods may require more careful feature selection and can become less effective as data complexity increases.
4. ANNs can be used as ML models and an ANN with multiple hidden layers is generally considered a Deep Learning model.



• Example of AI, ML, DL and ANN for a Specific Task

- ❖ **AI:** The overall goal is to make a computer recognize cats.
- ❖ **ML:** The computer learns the characteristics of cats from examples.
- ❖ **DL:** A deep neural network automatically learns features such as edges, shapes, eyes, ears and overall patterns.
- ❖ **ANN:** The connected neurons / layers perform the computations used to learn those patterns.
- ❖ **Note:** - Not every AI system uses ML and not every ML algorithm is Deep Learning.

- **Types of Machine Learning**

There are three major types:

1. Supervised Machine Learning
2. Unsupervised Machine Learning
3. Reinforcement Machine Learning

01. Supervised Learning

Type of Machine Learning where the model learns from labeled data in which both the input and the correct output are provided.

- **Main types of Supervised Machine Learning**

01. Classification: Used to predict a category or class. (a discrete / categorical value)

Ex: - Male or Female / True or False

02. Regression: Used to predict a continuous numerical value or quantity.

Ex: - Salary / Age / Price

- **Common Classification Algorithms**

1. Decision Tree Classifier
2. Random Forest Classifier
3. K - Nearest Neighbors (KNN)
4. Naive Bayes
5. Support Vector Machine (SVM)

- **Common Regression Algorithms**

1. Linear Regression
2. Polynomial Regression
3. Multiple Linear Regression

❖ **Important:** KNN and SVM can also be used for regression depending on the version used:

- **KNN** → **KNeighborsClassifier** / **KNeighborsRegressor**
- **SVM** → **SVC** / **SVR**

02. Unsupervised Machine Learning

Type of Machine Learning where the model learns patterns and relationships from data without labeled outputs. There is no correct answer / target column provided to the model.

- **Main types of Unsupervised Machine Learning**

01. Clustering: An unsupervised learning technique that groups similar data points together.

Ex: - Group customers based on their shopping behavior such as frequent buyers, occasional buyers and etc.

02. Association: An unsupervised learning technique that identifies important relationships patterns or rules between items or data points in a dataset.

Ex: - Customers who buy bread often also buy butter and uses this relationship for product recommendations.

- **Common Clustering Algorithms**

1. K - Means Clustering
2. Hierarchical Clustering
3. DBSCAN (Density - Based Spatial Clustering of Applications with Noise)

- **Common Association Algorithm**

1. Apriori - Used for finding frequent item sets and association rules.
2. FP - Growth (Frequent Pattern Growth) - Faster than Apriori for many large datasets.
3. Eclat - Uses a different approach to find frequent item sets efficiently.

03. Reinforcement Learning

Type of ML where an agent learns by interacting with an environment and receiving rewards or penalties for its actions.
Ex: - A robot needs to find its way from Room A to Room B.

1. Environment: The world or situation in which the agent operates.
2. Agent: The learner or decision - maker that interacts with the environment.
3. Action: Something the agent chooses to do.
4. Reward: Feedback given to the agent after an action. It tells the agent whether the action was good or bad.

- **Types of Reinforcement Learning**

1. Model - Based RL - The agent learns or uses a model of how the environment behaves.
2. Model - Free RL - The agent learns directly from experience without explicitly building a model.

- **Common RL Algorithms**

1. Q - Learning: Famous value - based RL algorithm.
2. SARSA - Another value - based algorithm.
3. Deep Q - Network (DQN) - Combines Q - learning with deep neural networks.

- **Types of Artificial Neural Network (ANN)**

1. CNN (Convolutional Neural Network): Type of neural network designed especially for image and spatial data.
2. RNN (Recurrent Neural Network): Type of neural network designed to process sequential or time - dependent data. Consider previous information when processing the current input.

- **Machine Learning (ML) vs Deep Learning (DL)**

01. ML: Usually requires feature extraction before training.
02. DL: Automatically learns and extracts features from raw data.

- **Applications of Deep Learning**

1. Healthcare: Medical image analysis and disease detection
2. Autonomous Vehicles: Object and road - sign detection
3. Computer Vision: Image and video recognition
4. Natural Language Processing (NLP): Language understanding, translation and chatbots

- **First Decide: AI, ML, DL or ANN**

Approach	When to use	Example
AI	Broad problem involving intelligent decision - making / reasoning.	Chess - playing system, expert system
ML	System learns patterns from data to make predictions / decisions.	Predict house price, classify emails
DL	Very large / complex data especially images, audio, video and NLP.	Face recognition, speech recognition
ANN	Neural - network - based solution; useful for complex non - linear patterns.	Handwritten digit recognition

- **Model Selection Criteria**

1. Supervised Machine Learning: The dataset has a target / output / label.
2. Unsupervised Machine Learning: There is no target / label and we want to discover patterns or groups.

3. <u>Decision Tree</u> - Classification. - Data is relatively easy to split using rules. - Interpretability is important. - Model that is easy to explain.	4. <u>Random Forest</u> - Classification. - Dataset has many features. - Better accuracy and more robust predictions. - A single decision tree may overfit.
5. <u>K - Nearest Neighbors (KNN)</u> - Dataset is small / medium. [Slower to Larger Ones.] - Similarity / distance makes sense. - Local patterns are important. <u>Avoid:</u> - Dataset is extremely large. - There are many irrelevant features. - Features are not properly scaled.	6. <u>Naive Bayes</u> - Classification problem. - Features are relatively independent. - Text classification is involved. Ex: - 1. Spam email detection 2. Sentiment classification 3. Document / News classification
7. <u>Support Vector Machine (SVM)</u> - Classification is required. - Classes need a clear boundary. - Dataset is small / medium. - High - Dimensional data exists.	8. <u>Linear Regression</u> - Supervised Learning. - Target is continuous numerical. - Relationship between input and output is approximately linear.
9. <u>Polynomial Regression</u> - Target is continuous. - Relationship between X and Y is curved / nonlinear. - A straight line does not fit the data well.	10. <u>Multiple Linear Regression</u> - Multiple input variables predicting one continuous numerical output.
11. <u>K-Means Clustering</u> - Divide data into K groups / clusters.	12. <u>Hierarchical Clustering</u> - Create a hierarchy of clusters. - Don't necessarily know the best number of clusters. - A dendrogram is useful.
13. <u>DBSCAN</u> - Clusters have irregular / non - circular shapes. - There may be noise / outliers. - No need to specify the number of clusters beforehand.	14. <u>PCA - Principal Component Analysis</u> - Used to reduce many features into fewer important components while retaining as much information / variance as possible.

- **K-Means vs Hierarchical vs DBSCAN**

K-Means	Hierarchical	DBSCAN
Need K	No fixed K initially	No K required
Good for spherical clusters	Hierarchical structure	Irregular shapes
Sensitive to outliers	Can be affected	Good at detecting noise
Uses centroids	Uses hierarchy	Uses density

- **Summary**

01. Types of Data Available

- **Image & Video Data** → CNN
- **Text & Speech Data** → RNN
- **Numerical / Tabular Data** → SVM, Logistic Regression, Decision Tree

02. Based on the Task

- **Classification** → SVM, Decision Tree, Random Forest
- **Regression** → Linear Regression, Polynomial Regression
- **Clustering** → K-Means, Hierarchical Clustering
- **Binary Classification** → Logistic Regression

- Accuracy, Recall, Precision and the Confusion Matrix are used to compare models and select the best one.

- **Supervised Machine Learning Technique 01: Regression**

“**Regression**” is a statistical and machine learning method used to predict or estimate a continuous (numeric) outcome based on one or more input variables.

- **Common Types of Regression**

01. **Linear Regression:** Fits a straight - line relationship between variables.
02. **Multiple Linear Regression:** Uses multiple input variables to make predictions.
03. **Polynomial Regression:** Models curved relationships.

1. **Linear Regression (LR)**

“**Linear Regression**” is a supervised machine learning algorithm used to predict a continuous numerical value from one or more input variables. [When to Use: Target / output is continuous. / Data Less => Linear Regression is better.]
Ex: -

1. Input: House size
2. Output: House price

- **Basic Equation**

$$\hat{y} = b_0 + b_1x$$

- x = Input / Independent Variable
- $\hat{y}$ = predicted output
- b_0 = intercept
- b_1 = slope/coefficient

- **Main Idea**

Linear Regression tries to find the **best - fitting straight line** through the data. The goal is to make the difference between the **actual values** and **predicted values** as small as possible.

[Method Name to Find Best Fitted Line: Least Square Method]

This difference is called the **residual / error**:

$$\text{Residual} = y - \hat{y}$$

2. Multiple Linear Regression (MLR)

“**Multiple Linear Regression**” is a regression technique used to predict a continuous numerical dependent variable using two or more independent variables assuming a linear relationship between them.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

3. Polynomial Regression [Non - Linear Regression]

“**Polynomial Regression**” is used when the relationship between X and Y is curved / non - linear rather than approximately straight.

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

- **Note**

A higher degree does not automatically mean a better model.

1. Degree too low → Underfitting
2. Degree appropriate → Good fit
3. Degree too high → Overfitting

- **Method to Compare Model Performance:** Evaluation Metrics like RMSE and R^2 .

- **RMSE - Root Mean Squared Error** [Elbow Plot - Lower Value]

- Measures how far the model's predictions are from the actual values.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of observations

- **R^2 (R-Squared)** [$0 \leq R^2 \leq 1$] [Elbow Plot - Higher Value]

- Measures how much of the variation in the target variable is explained by the regression model.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

1. $R^2 = 0$ → model explains none of the variation.
2. $R^2 = 1$ → model explains 100% of the variation.
3. **Higher R^2** → better fit.
4. **Test / Unseen Data** → Value can be negative!

- **Adjusted R² Formula**

$$\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

- ✓ R^2 = R - Squared
- ✓ n = Number of observations / data points
- ✓ p = Number of independent variables (predictors)
- Adjusted R^2 adds a penalty for additional predictors.

- **R² vs Adjusted R²**

- Adjusted R^2 penalizes unnecessary predictors while R^2 always increases when predictors are added.

$$\text{Adjusted } R^2 \leq 1$$

- **Maximum:** 1
- **Can be negative:** Yes
- **Closer to 1:** Better model
- **0 to 1:** Common range, but **not guaranteed**.

- **MAE Method**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value
- n = Number of Observations

$$\text{Lower MAE} \Rightarrow \text{Better model}$$

❖ MAE = 0 means perfect predictions.

- **MSE Formula**

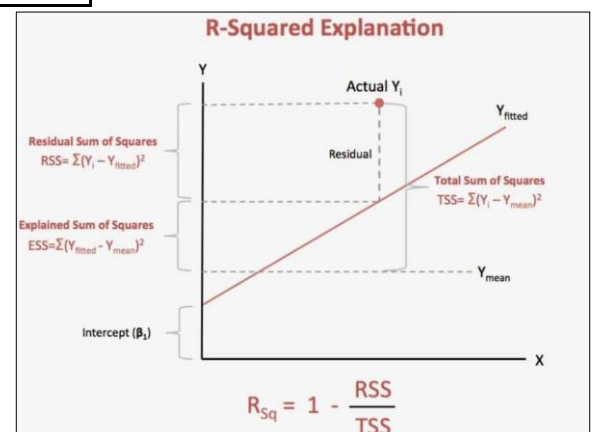
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{Lowest MSE} \Rightarrow \text{Best order}$$

- **Residual Analysis**

$$e_i = y_i - \hat{y}_i$$

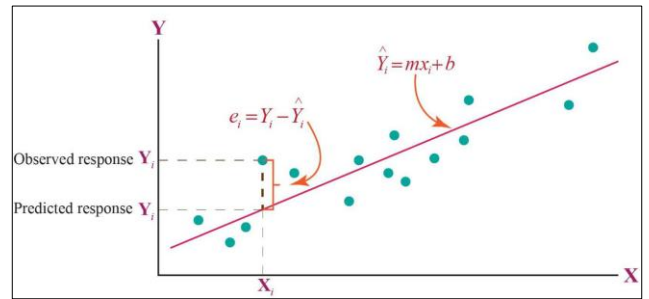
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- e_i = Residual / Error



• How to choose the order

1. Fit the model.
2. Calculate residuals.
3. Plot **Residuals vs Predicted Values**.
4. Look at the pattern.

- **Randomly scattered around 0: Good Model**
- **U / ∩ shape** → order may be too low
- **Increasing / decreasing spread** → non - constant variance
- **Clear pattern** → model is missing something



Best order = order with residuals randomly scattered around 0

• Cross-Validation

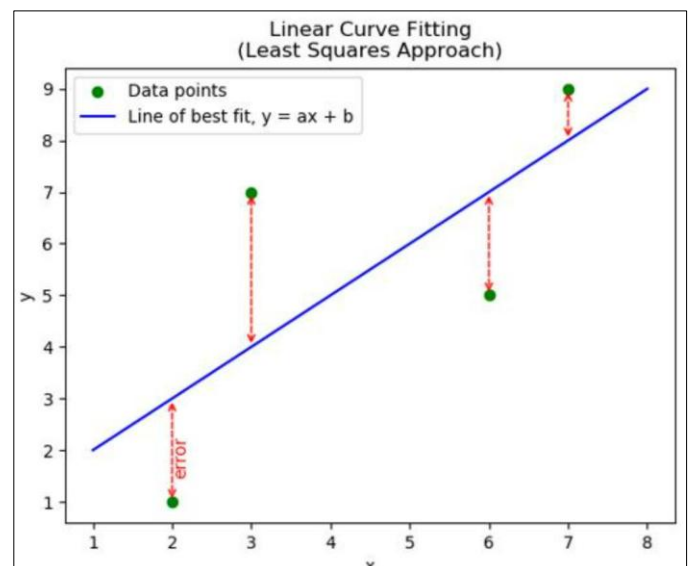
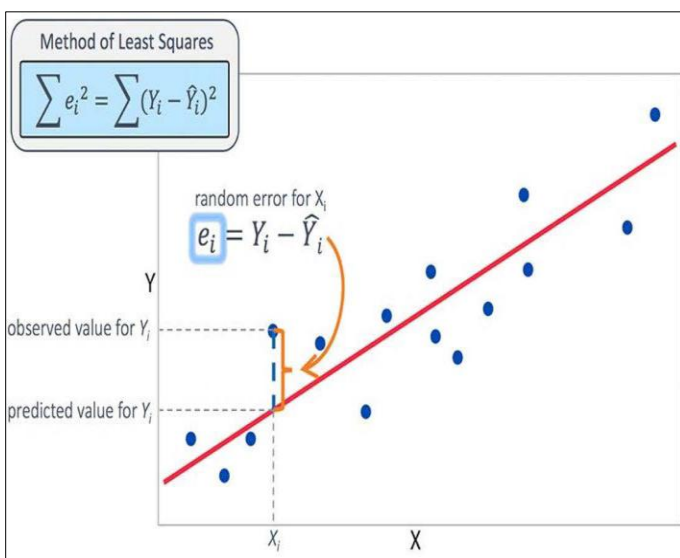
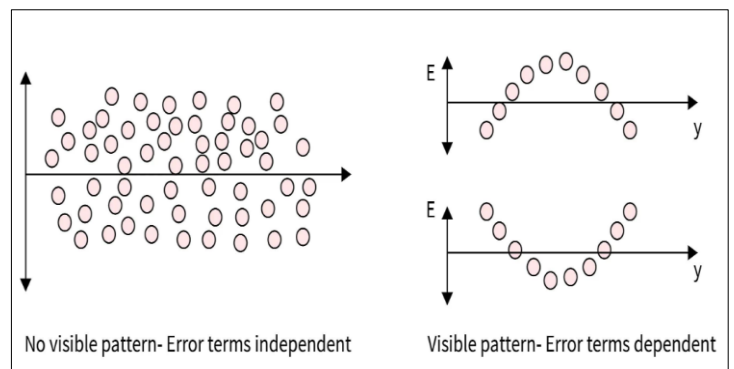
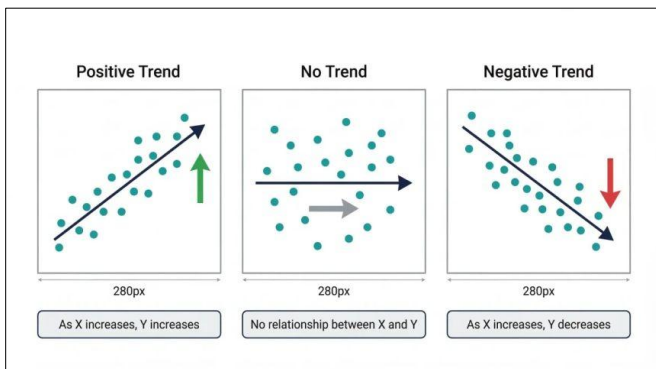
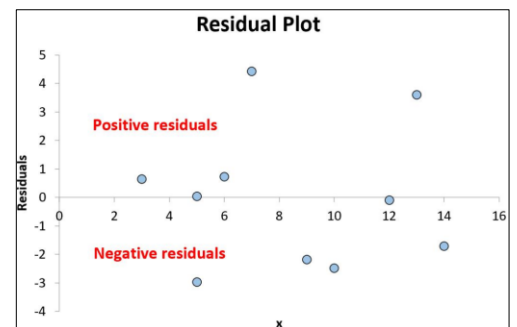
- CV tests how well each polynomial order performs on **unseen validation data**.

Best Order = order with lowest CV error

• Evaluation Methods in Regression Analysis

1. Residual Analysis
2. Cross - Validation
3. Mean Squared Error (MSE)
4. Mean Absolute Error (MAE)

• Extra Notes



• Standard Classification Techniques

1. K - Nearest Neighbors (KNN): - Classified Based on Closet Neighbors
2. Support Vector Machine (SVM): - Find the Best Separating Hyperplane
3. Random Forest: - Combine Many Trees for Better Accuracy
4. Decision Trees: - Make Decision Using Tree Rules
5. Naïve Bayes: - Use probability and assume feature independence

4. Logistic Regression [Classification Algorithm]

“**Logistic Regression**” is a **Supervised Machine Learning algorithm** mainly used for **Classification**. It predicts the **probability** that an observation belongs to a particular class.

Ex: -

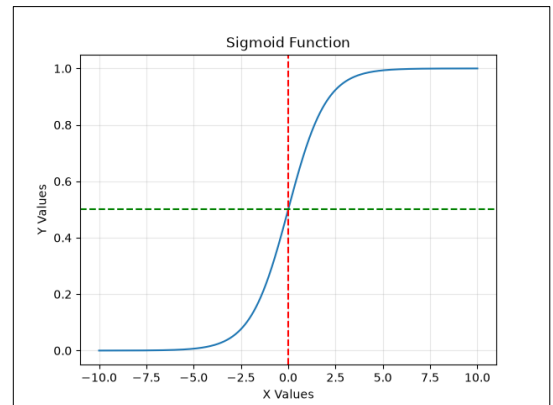
1. Predict whether a student will pass:
 - 0 → Fail ($P < 0.5$)
 - 1 → Pass ($P \geq 0.5$)

• Why Logistic Regression?

- Useful when the **target / output is categorical**.

• Sigmoid Function

- Uses the **Sigmoid function** to convert the model's output into a probability between **0 and 1**.



• Three Types of Logistic Regression

1. Binary Logistic Regression

[One or Many Features, Target: Two Classes]

- Used when there are 2 classes. [Ex: - Pass / Fail]
- Predicts One of Two Possible Classes.

2. Multinomial Logistic Regression

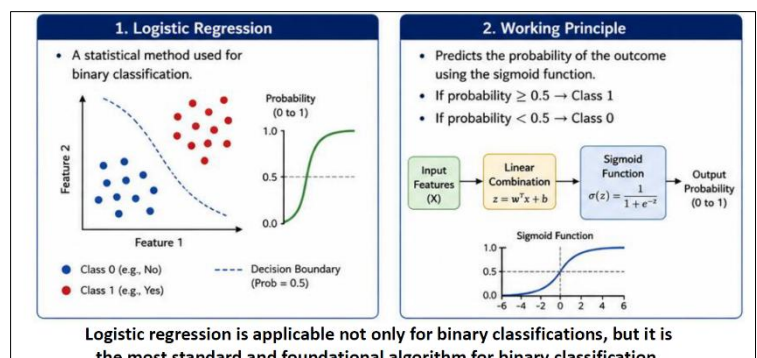
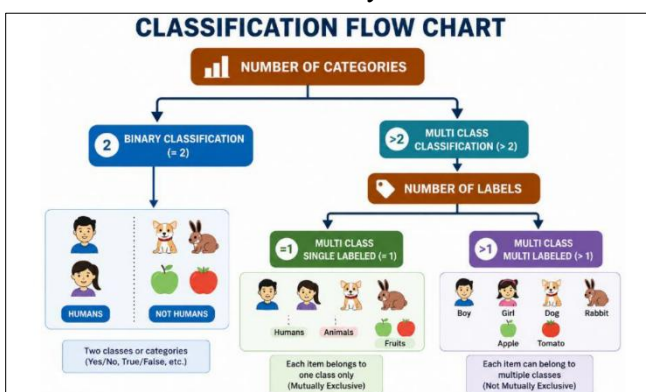
[One or Many Features, Target: More Classes]

- Used when there are more than 2 classes with no natural order. [Ex: - Car / Bus / Train]
- Predicts which type based on features. [Ex: - 0, 1, 2, ...]

3. Ordinal Logistic Regression

[One or Many Features, Target: More Classes with Rank]

- Used when there are more than 2 classes with a natural order. [Ex: - Low < Medium < High]
- No Standard Model from Python.



- **What is Train - Test Split?**

- **Train - Test Split** is the process of dividing a dataset into two separate parts:

1. **Training Data** → Used to train / learn the model.
2. **Testing Data** → Used to evaluate how well the trained model works on unseen data.

Ex: -

- **Total Records = 1000**
- **80% → Training data = 800 records**
- **20% → Testing data = 200 records**

- **Why Do We Use Train-Test Split?**

The main purpose is to check whether the model can **generalize to new / unseen data**. If we train and test using the same data, the model may simply memorize the training data. This can give a misleadingly high performance.

Ex: -

Imagine teaching a student 100 questions and then testing them using the **same 100 questions**. The student may get a very high score because they memorized the answers. Instead;

- ✓ **Training** → **Learn from known examples**
- ✓ **Testing** → **Check performance on new examples**

- **Important Attributes**

- 01.**x**: Contains the **input features / independent variables** used to make predictions.
- 02.**y**: Contains the **target / output / dependent variable** that the model needs to predict.
- 03.**test_size**: Determines **how much data should be used for testing**.
- 04.**train_size**: Specifies **how much data should be used for training**.
- 05.**random_state**: Controls the **randomness used when splitting the data**. [Split is reproducible.]
- 06.**shuffle**: Determines whether the dataset should be **randomly shuffled before splitting**.
- 07.**stratify**: Mainly useful for **classification problems**. It ensures that the **class proportions are approximately maintained** in both training and testing datasets. [stratify=y: When Data is imbalanced.]
- 08.**x_train**: Contains the **input features used to train the model**.
- 09.**x_test**: Contains the **input features used to test / evaluate the model**.
- 10.**y_train**: Contains the **target values corresponding to x_train**.
- 11.**y_test**: Contains the **actual target values corresponding to x_test**.

- **Rules**

1. Test Size must be 0 to 1.
2. Give most of the data to training and a small chunk to testing.
3. You set either “**test_size**” or “**train_size**” and not both.
4. Test set must be big enough to be meaningful.
5. For small datasets, use cross – validation.

5. KNN Algorithm [Classification Algorithm / May be used for Regression as well.]

K - Nearest Neighbors (KNN) is a supervised machine learning algorithm used for:

1. Classification → Predicting a category / class.
2. Regression → Predicting a numerical value.

KNN makes predictions by looking at the **K closest data points (neighbors)** to a new data point.

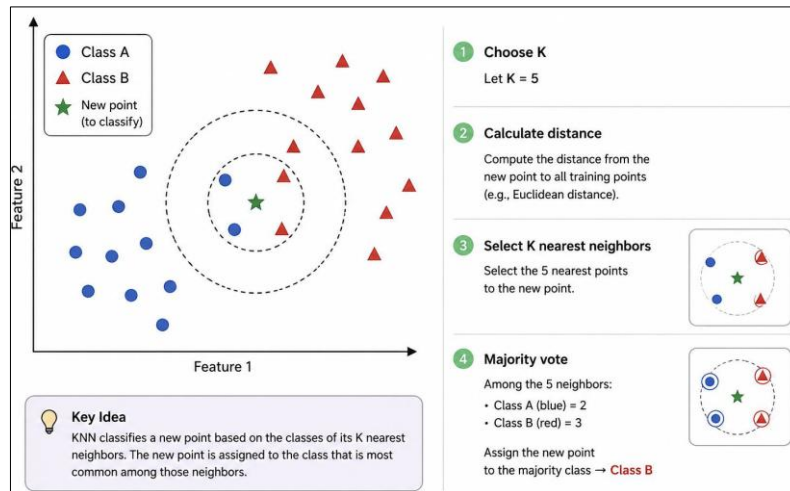
- **Workflow of the Algorithm [How KNN Works?]**

1. Choose the K Value. [Number of Neighbors]
2. For a new data point, calculate its distance to all training data points. [Ex: - Euclidean Distance]
3. Select the K nearest points.
4. For classification, Assign the class that is most common among those K neighbors. [Majority]

• Key Points

1. No explicit training phase.
2. Sensitive to feature scaling and choice of K.
3. Works well for small and medium datasets. [Slower for larger ones.]

• Graphical Summary



• Types of Learning

KNN is a **Supervised Learning (SL) algorithm**, because it learns from a dataset where the **target / output is already known**.

KNN Type	Purpose	Output
KNN Classification	Predict a class	Category
KNN Regression	Predict a value	Number

Ex: -

1. **Classification:** Is this email Spam or Not Spam?
2. **Regression:** Predict house price

• Why is it called K - Nearest Neighbors?

K → Number of neighbors considered.
Nearest → Points closest to the new data point.
Neighbors → Existing data points in the training dataset.

• Prediction Formula for a KNN Classification Problem

$\hat{y}$ = Most common class among K neighbors

• Prediction Formula for a KNN Regression Problem

$\hat{y}$ = Average Value among K neighbors

• Distance in KNN

KNN needs a way to determine which points are close to each other. The most common distance is **Euclidean Distance**.

For two points:

$P = (x_1, y_1)$ and $Q = (x_2, y_2)$

The Euclidean distance is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **Other Distance Measures**

Distance	Common Use
Euclidean	General numerical data
Manhattan	Grid - like distances
Minkowski	Generalized distance
Hamming	Categorical / Binary Data

- **Choosing the Value of K**

Small K [K = 1]	Large K [K = 50]
<u>Advantages:</u> 1. Very sensitive to nearby points. 2. Can capture local patterns. <u>Disadvantages:</u> 1. Sensitive to noise. 2. Can overfit.	<u>Advantages:</u> 1. Less sensitive to individual noisy points. <u>Disadvantages:</u> 1. Can ignore local patterns. 2. Can underfit.

- **Summary**

Small K → More flexible → Higher risk of overfitting
 Large K → Less flexible → Higher risk of underfitting

- **How to Choose the Best K**

For **classification**, compare:

1. Accuracy
2. Precision
3. Recall
4. F1 - Score

For **regression**, compare:

1. MAE
2. MSE
3. RMSE
4. R²

Choose the K that gives good validation performance.

- **Note**

- Do **not** simply choose the largest accuracy on the training data.

- Use a **validation set or cross-validation** to select K.

[Binary Classification: Use Odd K - Reduce the probability of tie. / Not Mandatory]

- **Feature Scaling in KNN [Since calculates distances.]**

1. <u>Standardization</u> [StandardScaler()] $z = \frac{x - \mu}{\sigma}$ <pre> scaler = StandardScaler() x_train = scaler.fit_transform(x_train) x_test = scaler.transform(x_test) </pre>	2. <u>Normalization</u> [MinMaxScaler()] [0 <= N <= 1] $X' = \frac{X - X_{min}}{X_{max} - X_{min}}$ <pre> scaler = MinMaxScaler() x_train = scaler.fit_transform(x_train) x_test = scaler.transform(x_test) </pre>
---	---

- **Important Points**

1. Lazy Learning Algorithm. [Training is easy. / Prediction may take a time.]
2. Non - Parametric Algorithm. [Uses the structure of the available data.]

• <u>Advantages of KNN</u> 1. Simple to understand. 2. Easy to implement. 3. Works for classification and regression. 4. No complicated training process. 5. Can model non - linear relationships.	• <u>Disadvantages of KNN</u> 1. Slow prediction with large datasets. 2. Sensitive to feature scaling. 3. Sensitive to irrelevant features. 4. Sensitive to noise. 5. Choosing K is important. 6. Performs poorly in very high dimensions.
---	--

- **Curse of Dimensionality** [Need Dimension Reduction / Feature Selection]

• 2 features → distance is meaningful. • 10 features → more difficult. • 100 features → distances may become less useful.

- **KNN Classifier - Important Parameters**

1. **n_neighbors**: Number of nearest neighbors used to make the prediction.
2. **weights**: Determines how much importance each neighbor gets.
 - a. **weights='uniform'**: All neighbors have equal importance.
 - b. **weights='distance'**: Closer neighbors get more importance. [Very Close - More / Distance - Less]
3. **algorithm**: Method used to find the nearest neighbors. ['auto', 'ball_tree', 'kd_tree', 'brute']
4. **leaf_size**: Controls the leaf size used by the tree - based algorithms. (ball_tree and kd_tree).
5. **p**: Controls the power used in the Minkowski distance.

- a. **Euclidean distance** (p = 2 / Common)

$$d = \sqrt{\sum (x_i - y_i)^2}$$

- b. **Manhattan distance** (p = 1)

$$d = \sum |x_i - y_i|$$

6. **metric**: Specifies the distance metric used to measure how close two points are.

Ex: -

Example: 'minkowski', 'euclidean', 'manhattan'

7. **metric_params**: Additional parameters required by a particular distance metric.
8. **n_jobs**: Controls how many CPU cores are used for neighbor - search operations.

06. **Decision Tree Algorithm** [Classification Algorithm / May be used for Regression]

- **What is a Decision Tree?** [Need 3 or More Classes in Target Variable.]

A Decision Tree is a supervised machine learning algorithm used for;

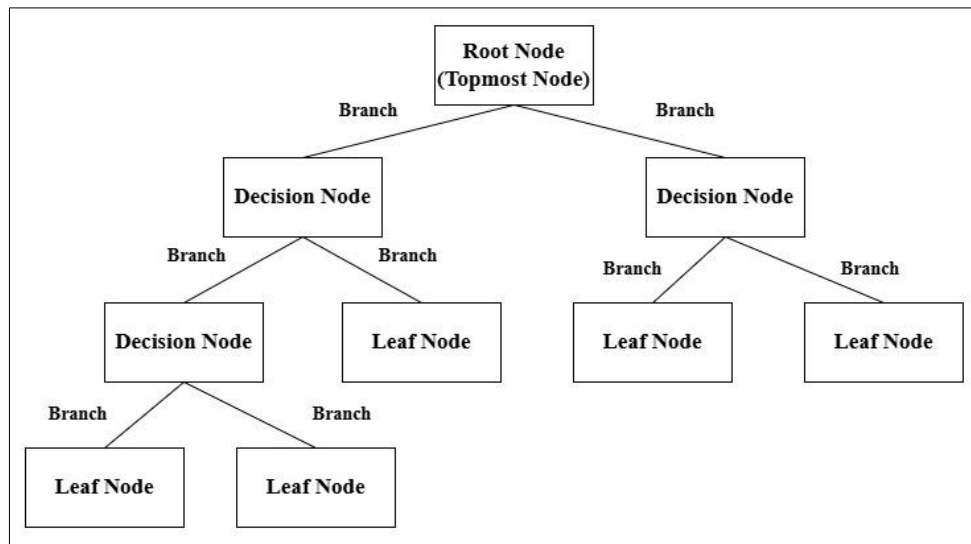
1. **Classification** → predicting a categorical output
2. **Regression** → predicting a numerical output

It works by repeatedly splitting the dataset based on features so that the resulting groups become as pure as possible. This is a method that makes decisions by asking a sequence of questions.

• Workflow of The Algorithm

1. Start with all data. [Root Node]
 - Algorithm looks at the full dataset first.
2. Feature Split.
 - Find the feature that best separates the data. [Ex: - Is age > 30?]
3. Split Data.
 - Based on the answer (Yes / No), the data is divided into groups.
4. Repeat Splitting.
 - Each Group is again split using new questions.
 - This continues until the data is well separated.
5. Make Final Decision. [Leaf Node]
 - Each end point gives a final prediction. [Class or Value]

• Basic Structure of a Decision Tree



• Important Terms [Upper Node: Parent / Lower Node: Child]

No	Term	Meaning
01.	Root Node	First / Top Decision
02.	Internal Node	A Feature - Based Decision [Decision Node]
03.	Branch	Result of a decision
04.	Leaf Node	Final prediction [End Point]
05.	Splitting	Dividing data into groups
06.	Depth	Number of levels in tree
07.	Pruning	Removing unnecessary branches

• Why Do We Split the Dataset?

The main objective is to create pure nodes.

1. Pure node: A node containing only one class. [Ex: - Yes, Yes, Yes, Yes / Purity = 100%]
 2. Impure node: A node containing different classes. [Ex: -Yes, Yes, No, No]
- The Decision Tree searches for a split that produces higher purity or lower impurity.

Feature Selection Matrices: Entropy + Information Gain / Gini Index / Log - Loss (Cross Entropy)

- **Decision Tree - Feature Selection Matrices**

1. **Entropy** [Lower Entropy = Better / More Pure]

- Measures the impurity or uncertainty or disorder in a node.
 - Tells how mixed the data is.
 - Try to reduce entropy at every split, then branch becomes “purer”.
 - Node contains only one class: Entropy = 0
 - Classes are mixed evenly: Entropy is high [Maximum disorder.]
- [Best Value = 0 / Worst Value = 1]

- ❖ **General Formula**

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- **Formula For Two Classes**

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

- ✓ p_+ = Proportion of positive examples
- ✓ p_- = Proportion of negative examples

- **Important values**

Situation	Entropy
All Yes	0
All No	0
50% Yes, 50% No	1
Pure node	0
Maximum uncertainty for binary classification	1

- **Information Gain** [Find Max Value]

- Tells us how much impurity / uncertainty is reduced after splitting.
- Choose the Split with highest information gains = Best Split

- **Formula**

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$\text{Information Gain} = \text{Parent Entrop} - \text{Weighted Child Entropy}$$

- **Gini Index** [Lower Gini = Better / Purer]

- Measure of impurity used by Decision Trees. [Measures how pure or how mix]
- It measures the probability of incorrectly classifying a randomly selected sample if it were labeled according to the class distribution of the node. [Best Value = 0 / Worst Value = 0.5]

$$\text{Gini} = 1 - \sum_{i=1}^c p_i^2$$

$$\text{Gini} = 1 - (p_+^2 + p_-^2)$$

- **Important values**

Situation	Gini
Pure Node [One Class]	0
50% - 50% binary node	0.5
Maximum binary impurity [Others it can be reached for 1]	0.5

- **Weighted Gini Index for a Split [Find Min Value]**

$$\text{Weighted Gini} = \sum_i \frac{N_i}{N} \text{Gini}_i$$

- **Log Loss [Logarithmic Loss / Cross - Entropy Loss] [Lower Log Loss - Better]**

- Evaluates the quality of predicted probabilities.
- Cares about how confident the model is.

- **Binary Classification Formula**

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- y_i = actual class (0 or 1)
- p_i = predicted probability of class 1
- N = number of samples

- **Complete Calculation Pattern [Entropy + Information Gain (IG)]**

1. Calculate parent entropy.

$$\text{Entropy(Parent)} = - \sum p_i \log_2 p_i$$

2. Calculate child entropy for every possible feature.

$$\text{Entropy(Child)} = - \sum p_i \log_2 p_i$$

3. Calculate weighted child entropy.

$$\text{WeightedEntropy} = \sum \frac{\text{ChildSize}}{\text{ParentSize}} \times \text{ChildEntropy}$$

4. Calculate Information Gain.

$$\text{IG} = \text{ParentEntropy} - \text{WeightedEntropy}$$

5. Compare all features.

$$\text{Highest IG} = \text{Best Feature}$$

- **Complete Calculation Pattern [Gini Index + Weighted Gini]**

1. Calculate Gini for each child.

$$\text{Gini} = 1 - \sum p_i^2$$

2. Calculate weighted Gini.

$$\text{WeightedGini} = \sum \frac{\text{ChildSize}}{\text{ParentSize}} \times \text{Gini}_{\text{child}}$$

3. Compare all splits.

Lowest Weighted Gini = Best Split

- **Decision Tree for Regression**

- Decision Trees can also predict numerical values.
- Choose split that reduce prediction error.
- Measures: MSE or MAE.

$$\text{MSE} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$$

- **Classification vs Regression Tree**

Term	Classification Tree	Regression Tree
Output	Class	Numerical value
Example	Disease / No Disease	House Price
Common split criteria	Gini, Entropy	MSE, MAE
Leaf prediction	Class/probability	Numerical prediction

- **Advantages of Decision Tree**

1. Easy to understand.
2. Easy to visualize.
3. Little preprocessing required.
4. Can handle numerical and categorical features.
5. Can be used for classification and regression.
6. Does not require feature scaling.
7. Can capture nonlinear relationships.
8. Easy to explain to non-technical users.

- **Disadvantages of Decision Tree**

1. Can overfit easily.
2. Small changes in data can create a different tree.
3. Deep trees become complicated.
4. Greedy splitting does not always produce the globally optimal tree.
5. Can favor certain types of splits / features.
6. Usually less stable than ensemble methods such as Random Forest.

- **How to reduce overfitting?**

1. max_depth
2. min_samples_split
3. min_samples_leaf
4. max_leaf_nodes
5. pruning
6. cross-validation

- **Important Decision Tree Parameters**

Parameter	Simple Meaning	Main Purpose
criterion	Method used to choose the best split. [Gini Index, Entropy, LogLoss]	Controls how nodes are split
max_depth	Maximum depth of the tree. [Important]	Controls tree complexity
min_samples_split	Minimum samples needed to split a node	Prevents unnecessary splitting
min_samples_leaf	Minimum samples required in a leaf	Prevents very small leaves
max_features	Number of features considered for a split	Controls feature selection
max_leaf_nodes	Maximum number of leaf nodes	Limits tree size
random_state	Fixes the random process	Gives reproducible results

07. **Random Forest** [Classification Algorithm / May be used for Regression]

- **What is Random Forest?**

“**Random Forest**” is a supervised machine learning algorithm that combines many Decision Trees to make a better and more stable prediction, improve accuracy and reduce overfitting.

- Logic: Create many decision trees during training using random subsets of the data and features

It can be used for:

1. Classification → predicting a category / class [Majority Vote=> Final Class]
2. Regression → predicting a numerical value [Average Predictions=> Final Value]

- **Why is it called "Random Forest"?**

Because it creates a forest = many Decision Trees and randomness is introduced when creating those trees. There are two important sources of randomness;

1. **Random samples of data.**
 - Each tree gets a different random sample of the training data using Bootstrap Sampling.
2. **Random subset of features.**
 - At each split, the tree considers only a random subset of features.

- **When should we use Random Forest?**

1. Dataset has many features.
2. Relationship between variables is complex / non - linear.
3. For a good predictive performance
4. Want a model that requires relatively little preprocessing
5. Single Decision Tree is overfitting
6. Want both classification and regression
7. Want some information about feature importance

- **Workflow of the algorithm**

1. **Bootstrap sampling. (Bagging)**
 - From the original dataset, multiple random samples are created with replacement. Each sample is used to train a separate decision tree.
2. **Random feature selection.**
 - At each split in a tree, only a random subset of features is considered rather than all features. This increases diversity among trees.
3. **Tree building.**
 - Each decision tree is grown independently.
4. **Aggregation of results.**
 - For classification, the final output is the majority vote from all trees.
 - For regression, the final output is the average of all tree predictions.

- **How does a Decision Tree decide a split? [Decision Tree Methods]**

- **For Classification; [Gini Index / Entropy / Log - Loss]**

1. **Gini Impurity**

$$\text{Gini} = 1 - \sum p_i^2$$

2. **Entropy**

$$\text{Entropy} = - \sum p_i \log_2(p_i)$$

- **Why Random Forest performs better than one Decision Tree**

- A Decision Tree can easily overfit.
- Random Forest combines many trees.
- Errors of individual trees can partially cancel each other.

- **What is Ensemble Learning?**

- Ensemble Learning means combining multiple models to produce a stronger model.

Ex: Random Forest is an ensemble of Decision Trees.

- **Important Random Forest Hyperparameters**

No	Hyperparameter	Meaning
01.	n_estimators	Number of trees.
02.	criterion	Measures the quality of a split. [Gini / Entropy / Log - Loss]
03.	max_depth	Maximum depth of each tree.
04.	max_features	Number / fraction of features considered when searching for a split.
05.	min_samples_split	Minimum number of samples required to split a node.
06.	min_samples_leaf	Minimum number of samples required in a leaf.
07.	bootstrap	Whether bootstrap samples are used.
08.	random_state	Controls randomness and makes results reproducible.
09.	max_samples	Number of samples to draw from X to train each base estimator. [When Bootstrap is true.]

- **Advantages of Random Forest**

1. High predictive performance.
2. Reduces overfitting compared with a single tree.
3. Handles non - linear relationships.
4. Works for classification and regression.
5. Little preprocessing.
6. Handles many features.
7. Feature importance.
8. Robust.

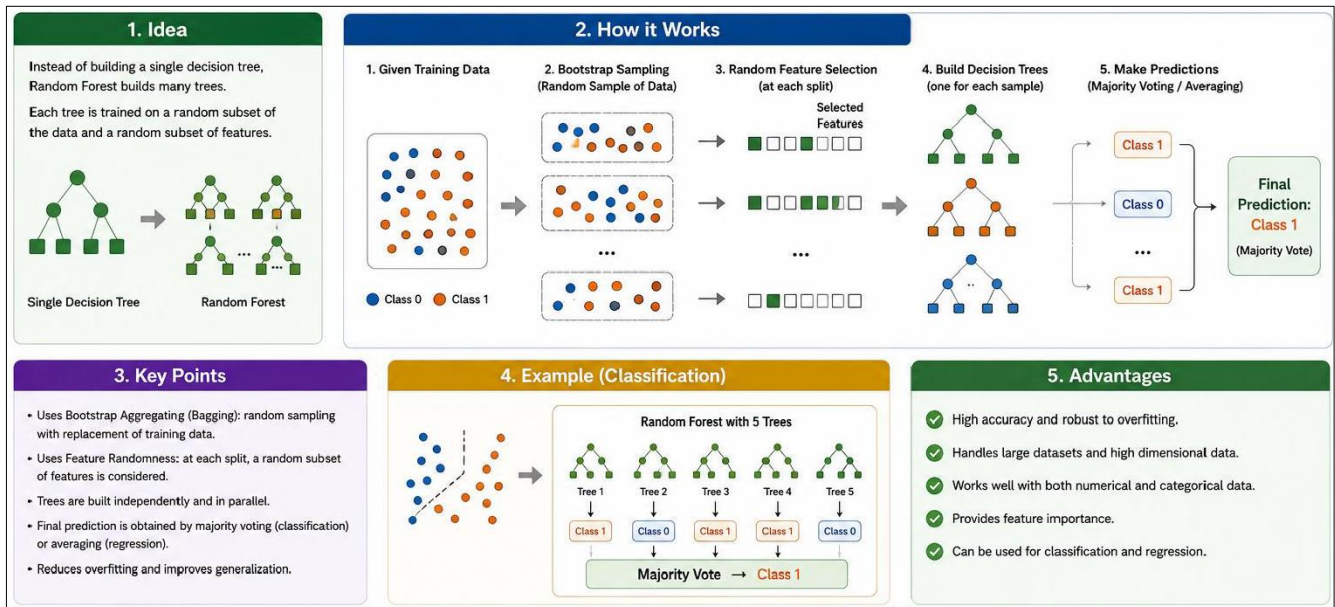
- **Disadvantages of Random Forest**

1. Computationally expensive.
2. Less interpretable.
3. Larger model.
4. Can be slower.
5. Feature importance can be misleading.

- **Algorithms that may Need Scaling**

1. KNN
2. SVM
3. Logistic Regression
4. Linear Regression

• Random Forest - Summary



• Parameter vs Hyperparameter

- **Parameter**: Learned by the model from data during training.
- **Hyperparameter**: Set by us before training to control how the model learns.

08. Naive Bayes Algorithm [Classification Algorithm]

• What is Naive Bayes?

“**Naive Bayes**” is a supervised machine learning classification algorithm based on “**Bayes’ Theorem**”. It is called “**naive**” because it assumes that **features are conditionally independent of each other given the class**. Commonly used for spam detection, text classification, sentiment analysis and etc.

• Probability vs Likelihood

01. Probability

- Question: “If I know the model / condition, what is the chance of the event?”

Ex: - $P(A | B)$ = probability of A when B is known.

02. Likelihood

- Question: “Given the observed data, how likely is a particular condition / model?”

Ex: - $P(B | A)$ are often interpreted as the likelihood of observing B when A is true.

• Conditional Probability

- $P(A | B)$: Probability of A given B.

$$P(A | B) \neq P(B | A)$$

Ex: -

1. $P(\text{Disease} | \text{Positive Test})$ → Probability that a person has disease given a positive test.
2. $P(\text{Positive Test} | \text{Disease})$ → Probability of getting a positive test given that the person has disease.

- **Bayes' Theorem**

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- $P(A | B) \rightarrow$ Posterior probability
- $P(B | A) \rightarrow$ Likelihood
- $P(A) \rightarrow$ Prior probability
- $P(B) \rightarrow$ Evidence / Marginal probability

- **Three Main Naive Bayes Methods**

1. **Gaussian Naive Bayes**

- Used when features are continuous numerical values and are assumed to follow a normal (Gaussian) distribution.
Ex: - Age, height, weight, temperature.

2. **Multinomial Naive Bayes**

- Used mainly for **count based data** especially **text classification**.
Ex: - Number of times words appear in email / document.

3. **Bernoulli Naive Bayes**

- Used when features are binary. [**0 / 1 or True / False**]
Ex: -
 1. Word present $\rightarrow$ 1
 2. Word absent $\rightarrow$ 0

- **Important Hyperparameters [Gaussian NB]**

Hyperparameter	Meaning	Important point
priors	Prior probability for each class	Can manually specify class probabilities
var_smoothing	Adds a small value to feature variance	Helps numerical stability

- **Important hyperparameters [Multinomial NB]**

Hyperparameter	Meaning	Important point
alpha	Smoothing parameter	Prevents zero probabilities
fit_prior	Whether to learn class prior probabilities	Usually, True
class_prior	Manually specify class probabilities	Optional

- **Important hyperparameters [Bernoulli NB]**

Hyperparameter	Meaning	Important point
alpha	Smoothing parameter	Prevents zero probabilities
binarize	Threshold used to convert values into 0 / 1	Important when input isn't already binary
fit_prior	Learn class prior probabilities	Usually, True
class_prior	Manually specify class priors	Optional

❖ **Self - Learning Topics [Out of Syllabus]**

01. XGBOOST
02. ICA (Independent Component Analysis)
03. PINN (Loss Function - First to Learn)
04. Agent Based Model
05. Handling Missing Values
06. Imbalanced Data

09. Support Vector Machine (SVM) Algorithm

- What is SVM?

Support Vector Machine (SVM) is a **supervised machine learning algorithm** mainly used for;

1. **Classification** → called **SVM (Support Vector Machine)**
2. **Regression** → called **SVR (Support Vector Regression)**

Its main goal is to find the **best decision boundary (hyperplane)** that separates different classes.

- Basic Idea

SVM tries to find a boundary that gives the **maximum margin** between the classes.

- **Margin** = distance between the decision boundary and the nearest data points from each class.
- **SVM chooses the hyperplane with the maximum possible margin.**

- Hyperplane

A **hyperplane** is the decision boundary used to separate classes. For two features;

$$w_1x_1 + w_2x_2 + b = 0$$

- ✓ **w** = weights
- ✓ **x** = input features
- ✓ **b** = bias

For higher dimensions, the boundary is still called a **hyperplane**.

- Support Vectors

Support vectors are the **data points closest to the decision boundary**. They are extremely important because they determine the position of the optimal hyperplane.

- **SVM = Support vectors + Maximum margin**

- Margin

The **margin** is the distance between the decision boundary and the closest points. SVM tries to;

Maximize the margin

A larger margin generally gives better generalization.

- Hard Margin vs Soft Margin

Type	Meaning
Hard Margin	No misclassification allowed
Soft Margin	Some misclassification is allowed

Soft margin is more commonly useful because real - world data may contain **noise and outliers**.

- C Parameter

- **C** controls the trade-off between **large margin** and **classification errors**.

C value	Effect
Large C	Strongly penalizes errors → narrower margin → may overfit
Small C	Allows more errors → wider margin → may generalize better

- **Kernel Trick**

Sometimes data cannot be separated using a straight line. SVM uses **kernels** to handle nonlinear relationships by effectively mapping data into a higher-dimensional space. Common kernels;

Kernel	Use
Linear	Data is approximately linearly separable
Polynomial	Polynomial relationships
RBF	Complex/nonlinear relationships
Sigmoid	Neural-network-like behavior; less commonly used

- **Important Kernel Parameters**

01. **kernel**: Specifies the kernel function. [**linear**, **rbf**, **poly**]
 02. **gamma**: It controls how far the influence of a training point reaches. [For RBF, Polynomial and Sigmoid]

Gamma	Effect
High gamma	Each point has a small influence → complex boundary → overfitting risk
Low gamma	Larger influence → smoother boundary → underfitting risk

03. **degree**: Used with the **polynomial kernel**. [High Degree => More Complex Boundary]

• <u>Advantages</u> 1. Works well in high-dimensional data. 2. Effective when there is a clear margin of separation. 3. Can handle non - linear data using kernels. 4. Works well with relatively small / medium datasets. 5. Uses only important boundary points. (Support Vectors)	• <u>Disadvantages</u> 1. Can be slow for very large datasets. 2. Sensitive to feature scaling. 3. Choosing the correct kernel and parameters can be difficult. 4. Less interpretable than a Decision Tree. 5. Performance can be affected by noisy / outlier - heavy data.
---	--

- **Cross - Validation Technique**

“**Cross - Validation (CV)**” is a technique used to check how well a machine learning model performs on different parts of the dataset. Instead of using only one train / test split, CV divides the training data into several parts called folds.

- **K-Fold Cross-Validation**

The most common method is K-Fold Cross-Validation. For example, with 5-fold CV;

Dataset
↓
[Fold 1] [Fold 2] [Fold 3] [Fold 4] [Fold 5]
Round 1 → Test Fold 1, Train on 2-5
Round 2 → Test Fold 2, Train on 1,3-5
Round 3 → Test Fold 3, Train on 1,2,4,5
Round 4 → Test Fold 4, Train on 1-3,5
Round 5 → Test Fold 5, Train on 1-4

Then the 5 scores are averaged.

- **Why use Cross-Validation?**

1. Gives a more reliable estimate of model performance.
2. Uses different portions of the data for testing.
3. Helps compare different models.
4. Helps detect whether a model performs consistently.

10. K - Means [Clustering Algorithm / Unsupervised Learning Technique]

• What is K - Means?

“**K - Means**” is an unsupervised machine learning algorithm used for clustering. It divides data into K groups (clusters) where data points in the same cluster are as similar as possible. [**K = number of clusters**]

- Unsupervised = No target / Output label is given.
- Mainly used for grouping similar data

• When to Use K-Means?

1. You have unlabeled data.
2. You want to group similar observations.
3. The number of clusters can reasonably be specified or estimated.
4. Clusters are roughly compact / spherical.
5. Numerical features are available.

• Important Terms

Term	Meaning
K	Number of clusters
Centroid	Center / Mean point of a cluster
Cluster	Group of similar data points
Distance	Measures how close a point is to a centroid
Inertia	Sum of squared distances from points to their nearest centroid

• How K - Means Works

01. Choose K: Decide the number of clusters.
02. Initialize centroids: Select K initial centroid positions.
03. Assign points: Each data point is assigned to the nearest centroid. [Euclidean Distance]
04. Recalculate centroids.
05. Calculate the mean of all points in each cluster.
06. Repeat until the centroids / assignments stop changing significantly.

• Euclidean Distance [The point is assigned to the centroid with the smallest distance.]

For two points:

$$A(x_1, y_1), B(x_2, y_2)$$

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For multiple features:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

How to Choose the Best K? => Elbow Method => Plot between K Values vs Inertia Values

- **Inertia**

“**Inertia**” measures how close data points are to their cluster centroids.

$$\text{Inertia} = \sum ||\mathbf{x}_i - \mathbf{c}_j||^2$$

- x_i = Data point
- c_j = Nearest centroid

- **Silhouette Score**

It measures;

1. How similar a point is to its own cluster [**Inter Cluster Distance** => Low]
2. How different it is from other clusters [**Intra Cluster Distance** => High]

$$-1 \leq \text{Silhouette Score} \leq 1$$

- **Interpretation**

Score	Meaning
Near 1	Very good clustering
Near 0	Overlapping clusters
Negative	Point may be in the wrong cluster

- **Advantages**

1. Simple and easy to understand.
2. Easy to implement.
3. Fast for large datasets.
4. Works well when clusters are clearly separated.
5. Easy to visualize.
6. Computationally efficient.

- **Disadvantages**

1. Must choose K.
2. Sensitive to initial centroid positions.
3. Sensitive to outliers.
4. Sensitive to feature scaling.
5. Can converge to a local optimum.
6. Doesn't work well for irregularly shaped clusters.
7. Results can vary depending on initialization.

- **Important Hyperparameters**

No	Hyperparameter	Meaning
01.	n_clusters	Number of clusters. (K)
02.	init	Method used to initialize centroids.
03.	n_init	Number of times K-Means is run with different centroid initializations
04.	max_iter	Maximum number of iterations for one run
05.	tol	Tolerance used to determine convergence
06.	random_state	Controls random initialization
07.	algorithm	Algorithm used to calculate distances/perform K-Means

11. **Hierarchical Clustering** [ML => Unsupervised => Clustering]

“**Hierarchical Clustering**” is an **unsupervised learning algorithm** that groups similar data points into clusters and creates a **hierarchy (tree) of clusters**. It is commonly represented using a **dendrogram**. It finds groups based on **similarity or distance**.

- **When to Use Hierarchical Clustering?**

1. You **do not** have labelled data.
2. You want to find **natural groups** in the data.
3. You want to understand the **relationship between clusters**.
4. You want a **dendrogram** to see how clusters are formed.
5. You don't know the exact number of clusters beforehand.

- **Main Types**

There are two approaches;

A. Agglomerative Clustering [Bottom - Up Approach]

1. Start with every data point as an individual cluster.
2. Find the two closest clusters.
3. Merge them.
4. Repeat until all points form one cluster.
5. Cut the dendrogram at a suitable level to obtain the required clusters.

B. Divisive Clustering [Top-down approach]

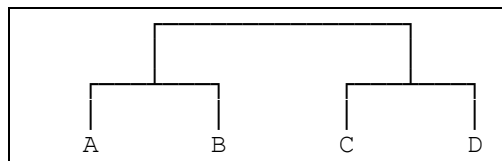
1. Start with all data points in one cluster.
2. Split the cluster into smaller clusters.
3. Continue splitting until individual clusters are obtained.

- **Dendrogram**

A **dendrogram** is a tree - like diagram that shows how clusters are merged or divided.

1. **X-axis:** Data points
2. **Y-axis:** Distance / dissimilarity

A **large vertical distance** between merges can indicate a good place to cut the tree.



- **Linkage Methods**

Linkage determines the distance between two clusters.

Linkage	Meaning
Single	Distance between the closest points
Complete	Distance between the farthest points
Average	Average distance between points
Ward	Minimizes increase in within-cluster variance

- **Advantages**

1. No need for labelled data.
2. Can visualize clusters using a **dendrogram**.
3. Does not require choosing the number of clusters.
4. Shows the **hierarchical relationship** between data points.

- **Disadvantages**

1. Can be computationally expensive for large datasets.
2. Sensitive to the choice of distance and linkage method.
3. Once clusters are merged, they generally **cannot be undone** in agglomerative clustering.
4. Outliers can affect the clustering.

- **RandomizedSearchCV & GridSearchCV [Hyperparameter Tuning]**

- **What is Hyperparameter Tuning?**

A machine - learning model has settings that we choose before training. These are called “**Hyperparameters**”. We try different values and find the combination that gives the best cross-validation performance.

- **GridSearchCV**

GridSearchCV tries every possible combination of the hyperparameters you provide. It tests every combination and each combination is evaluated using 5-fold cross-validation.

- **RandomizedSearchCV**

RandomizedSearchCV does not test every combination. Instead, it randomly selects a specified number of combinations.

- **GridSearchCV vs RandomizedSearchCV**

Feature	GridSearchCV	RandomizedSearchCV
Tests	Every combination	Random combinations
Speed	Usually, slower	Usually, faster
Computational cost	High	Lower
Good for	Small parameter space	Large parameter space
Control	Complete/exhaustive	Random sampling
Important parameter	param_grid	param_distributions
Number of trials	Automatically all	n_iter

- **Data Cleaning Methods**

These are **image - processing filters** mainly used for **noise reduction / smoothing**.

Filter	Main purpose	Best for	Output
MA (Moving Average)	Smooth image	General/random noise	Smooth but can blur edges
Median Filter	Remove noise	Salt-and-pepper noise	Preserves edges better
Gaussian Filter	Smooth image	Gaussian/random noise	Smooth, natural-looking blur

- **Moving Average (MA) Filter**

The **Moving Average (MA) filter** replaces each pixel with the **average of its neighboring pixels**.

• <u>Why use it?</u> 1. Reduces random noise 2. Smooths an image 3. Simple to understand and implement	• <u>Disadvantage</u> 1. It can blur edges, because it treats all neighboring pixels equally.
---	---

- **Median Filter [Salt - Pepper Filter]**

The **Median Filter** replaces a pixel with the **median value** of its neighboring pixels.

- **Why use it?**

1. Especially good for **salt-and-pepper noise**.
 - Salt → very bright pixels
 - Pepper → very dark pixels
2. It generally **preserves edges better than MA**.

- **Gaussian Filter**

The **Gaussian filter** performs smoothing using a **Gaussian distribution**. Unlike MA, neighboring pixels do **not all have equal importance**. Pixels closer to the center receive **higher weights**.

- **Why use it?**

1. Reduce noise.
2. Smooth images.
3. Remove small details.
4. Commonly used before edge detection.
5. Useful as preprocessing in computer vision.

- **Error Bar Plot**

An **error bar plot** is a graph used to show;

1. The **main value**. (such as mean)
2. The **uncertainty or variation** around that value.

How much the value may increase or decrease.

- **When to use an Error Bar Plot?**

1. Mean $\pm$ standard deviation
2. Mean $\pm$ standard error
3. Confidence intervals
4. Measurement uncertainty
5. Variation in experimental results

- **Important parameters**

Parameter	Meaning
x	X-axis values
y	Main/mean values
yerr	Size of error
fmt='o'	Display means as a circle
capsize=5	Size of the small horizontal caps

12. DBSCAN Algorithm

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an **unsupervised learning** clustering algorithm. It groups data points based on **density** and can identify **outliers / noise**.

- **Main idea:** Points that are close together and have enough neighboring points form a cluster.

- **When to use DBSCAN?**

1. You **do not have labels** → Unsupervised Learning
2. You want to find **clusters automatically**
3. Clusters may have **irregular/non-circular shapes**
4. Your dataset contains **outliers/noise**
5. You don't know the number of clusters beforehand

- **Important DBSCAN Parameters**

01. **eps**: Maximum distance around a point to search for neighboring points.

- Small **eps** → fewer points considered neighbors.
- Large **eps** → more points considered neighbors.

02. **min_samples**: Minimum number of points required in the **eps** neighborhood to form a dense region.

- **Three Types of Points [Noise = -1]**

01. **Core Point**: Has at least “**min_samples**” points within **eps**.

02. **Border Point**: Not a core point itself, but is close to a core point.

03. **Noise Point**: Does not belong to any cluster.

- **Principal Component Analysis (PCA)**

“PCA (Principal Component Analysis)” is an **unsupervised dimensionality reduction technique**. It converts many original features into a smaller number of new features called **Principal Components (PCs)** while trying to preserve as much information (variance) as possible.

- **Why use PCA?**

1. Reduce the number of features.
2. Remove redundancy / correlation.
3. Make ML models faster.
4. Reduce storage requirements.
5. Help visualize high - dimensional data.
6. Reduce noise in some cases
7. Avoid the **curse of dimensionality**

$$Z = \frac{X - \mu}{\sigma}$$

$$X_{\text{centered}} = X - \bar{X}$$

$$\text{Cov}(X, Y) = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{n - 1}$$

$$\text{Cov}(X) = \frac{1}{n - 1} X^T X$$

- **Important idea**

PCA looks for directions in the data where the **variance is maximum**. The components are ranked;

1. **PC1** → **highest variance**
2. **PC2** → **second-highest variance**
3. **PC3** → **third-highest variance**

$$Z = XW$$

- Principal components are mutually orthogonal (uncorrelated).

- **PCA Steps**

1. **Original Data**
2. **Standardize / Scale Data**
3. **Center the Data**
4. **Calculate Covariance Matrix**
5. **Calculate Eigenvalues & Eigenvectors**
6. **Sort Eigenvalues**
7. **Select Principal Components**
8. **Transform Data**

$$\text{Explained Variance Ratio} = \frac{\lambda_i}{\sum \lambda}$$

- **Eigenvectors**: Eigenvectors represent the **directions** of the principal components.
- **Eigenvalues**: Eigenvalues represent the **amount of variance/information** captured in those directions.

- **Confusion Matrix**

A **Confusion Matrix** is a table used to evaluate a **classification model**. It compares the **Actual value** with the **Predicted value**.

- T - Prediction is Correct
- F - Prediction is Wrong
- P - Model predicts Positive
- N - Model predicts Negative

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

1. **TP - True Positive**

- **Actual:** Positive
 - **Predicted:** Positive
 - Model is **correct**.
- ✓ **Example:** A patient has a disease → Model predicts **Disease**.

2. **TN - True Negative**

- **Actual:** Negative
 - **Predicted:** Negative
 - Model is **correct**.
- ✓ **Example:** A patient does not have a disease → Model predicts **No Disease**.

3. **FP - False Positive** [False Alarm]

- **Actual:** Negative
 - **Predicted:** Positive
 - Model is **wrong**.
- ✓ **Example:** A patient does not have a disease → Model predicts **Disease**.

4. **FN - False Negative** [Missed Case]

- **Actual:** Positive
 - **Predicted:** Negative
 - Model is **wrong**.
- ✓ **Example:** A patient has a disease → Model predicts **No Disease**.

- **Classification Evaluation Metrics**

Metric	Formula	Meaning
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	How many predictions are correct overall
Precision	$TP / (TP + FP)$	Of all cases predicted Positive , how many were actually positive?
Recall	$TP / (TP + FN)$	Of all actual Positive cases, how many did the model find?
F1 Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	Balance between Precision and Recall

- **Important Definitions**

Term	Simple Definition	Example
Feature Selection	Choosing the most useful existing features from a dataset and removing unnecessary features.	From Age, Height, Weight, Name → select Age, Height, Weight
Feature Extraction	Creating new features from existing features to represent the data better.	PCA combines many features into a few new components
Dimensionality Reduction	Reducing the number of features/dimensions while trying to keep important information.	Reducing 10 features → 3 features using PCA
Data Cleaning	Finding and fixing/removing incorrect, missing, duplicate, or inconsistent data .	Filling missing age values or removing duplicate rows
Feature Scaling	Bringing numerical features to a similar scale/range so one feature doesn't dominate others.	Age: 20–60 and Salary: 20,000–200,000 → scale them
Overfitting	Model learns the training data too well , including noise, and performs poorly on new data.	Training accuracy = 99%, Testing accuracy = 70%
Underfitting	Model is too simple to learn the important patterns in the data.	Training accuracy = 60%, Testing accuracy = 58%

- **SVD (Singular Value Decomposition)**

SVD is a mathematical technique used to **break a matrix into three smaller matrices**. In ML, SVD can help **reduce a large number of features into fewer important features**.

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- $\mathbf{A}$ = original matrix
- $\mathbf{U}$ = left singular vectors
- $\mathbf{\Sigma}$ (**Sigma**) = singular values
- $\mathbf{V}^T$ = transpose of $\mathbf{V}$

- **Why is SVD used?**

1. **Dimensionality reduction**
2. Data compression
3. Noise reduction
4. Feature extraction
5. Recommendation systems
6. Image processing

- **Simple Calculation**

Suppose;

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

We want;

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- **Step 1: Calculate $\mathbf{A}^T \mathbf{A}$**

$$\mathbf{A}^T = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

Therefore;

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 9 & 0 \\ 0 & 4 \end{bmatrix}$$

- **Step 2: Find eigenvalues**

The eigenvalues are;

$$\lambda_1 = 9, \lambda_2 = 4$$

- **Step 3: Find singular values**

Take the square root;

$$\sigma_1 = \sqrt{9} = 3$$

$$\sigma_2 = \sqrt{4} = 2$$

So;

$$\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

- **Step 4: Find U and V**

For this very simple matrix;

$$U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Therefore:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$